

NVE engine parity: LOREM and PET, at the neck of the metatomic hourglass

`metawork/atomistic-cookbook/examples/metatomic-hourglass` is an upstream cookbook recipe demonstrating that a model exported to the common **metatomic** format runs identically across engines (ASE, LAMMPS, GROMACS, i-PI, TorchSim) -- an NVE trajectory started from rest should be the same deterministic trajectory everywhere, to sub-meV. This notebook applies that same check to this project's own SN2 models, `sn2-matched-lorem` and `sn2-matched-pet` (see `model_registry.py`, notebooks 07 / 09), on four of the five engines: **ASE** and **i-PI** throughout, plus **LAMMPS** (`metawork/lammps`, branch `metatomic`, `PKG_ML-METATOMIC`, section 3) and **GROMACS** (`metawork/gromacs`, branch `metatomic`, `GMX_METATOMIC=TORCH`, section 4) -- both built specifically for this notebook. Only TorchSim still isn't installed -- see `hourglass_engines.py`'s docstring.

Unlike the upstream recipe's neutral ethanol molecule in a large periodic box, our reference structure (`data/sn2/sn2.xyz`, frame 0, reused from notebooks 07 / 09) is a **charged** (net -1), **non-periodic** 6-atom complex, $\text{CH}_3\text{F}\cdots\text{I}^-$. That difference turns out to matter: it is the reason this notebook does *not* end with a clean "everything matches" result, and why section 1 below hunts for the box size that comes closest.

Setup

Same protocol as the upstream recipe: `VelocityVerlet / nve` motion, a 0.5 fs timestep, 100 steps, zero initial velocities -- so every engine should trace out the same deterministic 101-snapshot trajectory. ASE runs the structure as-is (non-periodic, `pbcs=False`); i-PI always treats its cell as periodic, so it needs an explicit vacuum box (`hourglass_engines.run_ipi`'s `cell` argument). `DEFAULT_CELL` below is not an arbitrary choice -- section 1 scans it and picks the largest size that still runs in a practical time (see there for why).

```
In [ ]: import sphericart.torch # pre-import before loading the model
import ase.io

from hourglass_engines import run_ase, run_ipi, DEFAULT_CELL
from model_registry import MODELS_BY_KEY

atoms0 = ase.io.read("sn2_frame0.xyz")
model_paths = {
    key: str(MODELS_BY_KEY[key]["dir"] / "model.pt")
    for key in ["sn2-matched-lorem", "sn2-matched-pet"]
}
```

```
print(model_paths)
print(f"default i-PI vacuum cell: {DEFAULT_CELL:g} Å")
```

1. Searching for the box size that minimizes disagreement

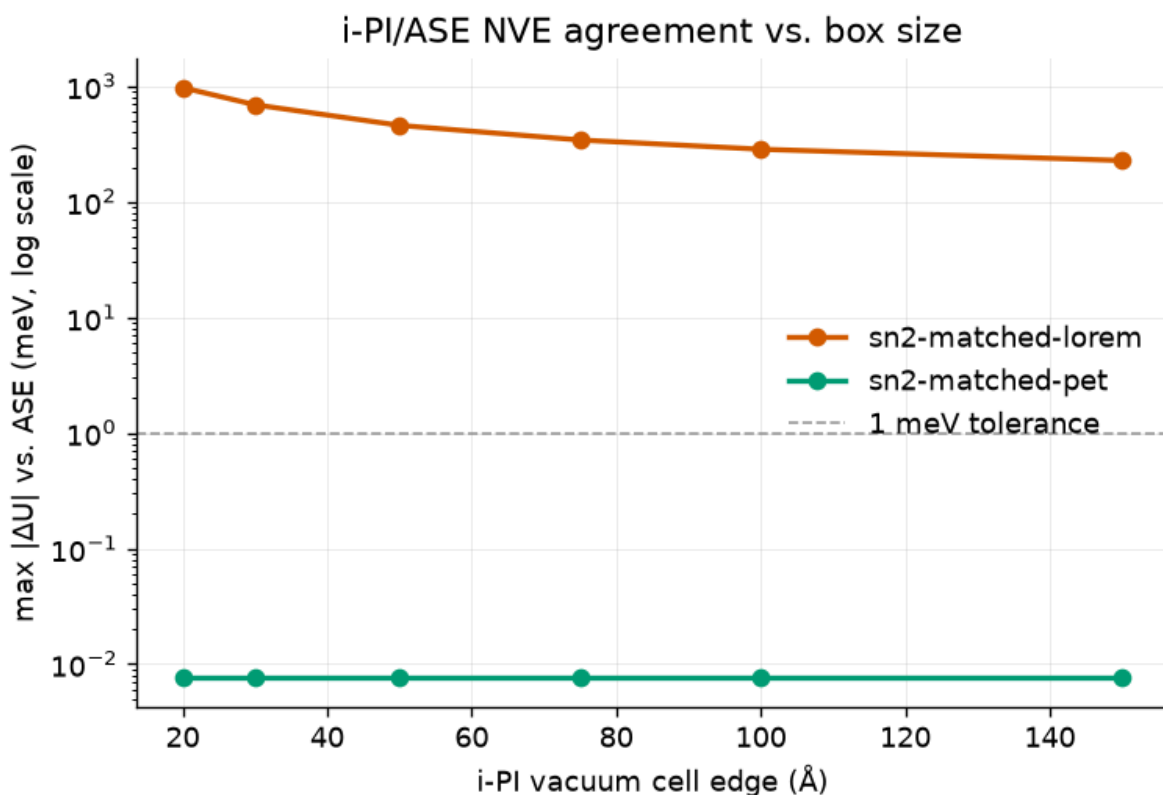
i-PI always treats its cell as periodic; ASE never does. For a genuine long-range model on a **charged** (net -1) structure, that's expected to matter -- periodic images of the net charge interact across the box -- and the effect should shrink as the box grows, since the images move further away. We scan the vacuum-cell edge and look for the size that minimizes the max $|\Delta U|$ against the ASE reference, for both models.

```
In [ ]: import numpy as np

BOX_SIZES = [20.0, 30.0, 50.0, 75.0, 100.0, 150.0]
ase_energies = {
    model_name: np.array(run_ase(model_path, atoms0, ensemble="nve")[1])
    for model_name, model_path in model_paths.items()
}
boxscan = {model_name: [] for model_name in model_paths}
for model_name, model_path in model_paths.items():
    for box in BOX_SIZES:
        _t, e = run_ipi(model_path, atoms0, ensemble="nve", cell=box)
        max_delta_meV = 1000 * np.abs(np.array(e) - ase_energies[model_name])
        boxscan[model_name].append((box, max_delta_meV))
```

```
In [ ]: import matplotlib.pyplot as plt

MODEL_COLORS = {"sn2-matched-lorem": "#D55E00", "sn2-matched-pet": "#009E73"}
fig, ax = plt.subplots(figsize=(6.5, 4.5))
for model_name, rows in boxscan.items():
    boxes, deltas = zip(*rows)
    ax.semilogy(boxes, deltas, "o-", color=MODEL_COLORS[model_name], linewidth=2)
ax.axhline(1.0, color="0.6", linestyle="--", linewidth=1, label="1 meV tolerance")
ax.set_xlabel("i-PI vacuum cell edge (Å)")
ax.set_ylabel("max  $|\Delta U|$  vs. ASE (meV, log scale)")
ax.set_title("i-PI/ASE NVE agreement vs. box size")
ax.legend()
plt.show()
```



model	box (Å)	max $ \Delta U $ (meV)
sn2-matched-lorem	20	974.6627
sn2-matched-lorem	30	693.1705
sn2-matched-lorem	50	459.8426
sn2-matched-lorem	75	343.3971
sn2-matched-lorem	100	285.5949
sn2-matched-lorem	150	228.1694
sn2-matched-pet	20	0.0076
sn2-matched-pet	30	0.0076
sn2-matched-pet	50	0.0076
sn2-matched-pet	75	0.0076
sn2-matched-pet	100	0.0076
sn2-matched-pet	150	0.0076

PET is unaffected by box size (spread over the whole scan: 0.0076 meV, engine roundoff) -- expected for a short-range model (empirical cutoff ~ 5.5 Å, notebook 12): once the vacuum cell clears the interaction range, periodicity cannot matter.

LOREM improves monotonically but slowly as the box grows: 20 Å gives 974.7 meV, 150 Å gives 228.2 meV -- moving in the expected direction (larger box, more distant periodic images, smaller effect), but nowhere near the upstream recipe's 1 meV bound within this range.

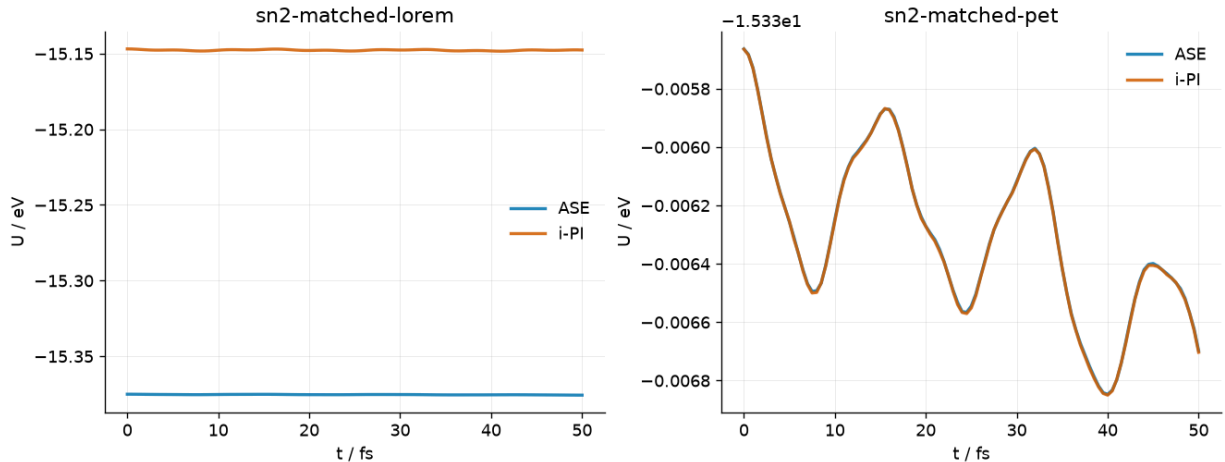
We tried pushing further (200, 300, 500, 750, 1000 Å) to see how far the minimum could be pushed down. i-PI's Ewald k-space cost grows far faster than the $\sim 1/L$ error

decay: a single 100-step run at 300 Å did not finish within 180 s, against a few tens of seconds per run anywhere in the 20-150 Å range above (RAM climbing past 17 GB before we stopped it). **150 Å is therefore the largest box size in this scan that stays practical to run** -- not a converged value, just the best one actually reachable here -- so `hourglass_engines.DEFAULT_CELL` is set to it.

2. ASE vs. i-PI at the chosen box size (150 Å)

```
In [ ]: trajectories = {}
for model_name, model_path in model_paths.items():
    ase_t, ase_e = run_ase(model_path, atoms0, ensemble="nve")
    ipi_t, ipi_e = run_ipi(model_path, atoms0, ensemble="nve") # DEFAULT_CELL
    trajectories[model_name] = {"ASE": (ase_t, ase_e), "i-PI": (ipi_t, ipi_e)}
```

```
In [ ]: ENGINE_COLORS = {"ASE": "#0072B2", "i-PI": "#D55E00"}
fig, axes = plt.subplots(1, 2, figsize=(11, 4.2), constrained_layout=True, sharey=True)
for ax, (model_name, engines) in zip(axes, trajectories.items()):
    for engine_name, (t, e) in engines.items():
        ax.plot(t, e, label=engine_name, color=ENGINE_COLORS[engine_name], lw=2)
    ax.set_title(model_name)
    ax.set_xlabel("t / fs")
    ax.set_ylabel("U / eV")
    ax.legend()
plt.show()
```



model	engine	max $ \Delta U $ (meV)	status
sn2-matched-lorem	ASE	0.0000	OK
sn2-matched-lorem	i-PI	228.1694	MISMATCH
sn2-matched-pet	ASE	0.0000	OK
sn2-matched-pet	i-PI	0.0076	OK

PET matches to noise level (0.0076 meV). LOREM still doesn't (228.2 meV , ~228x the upstream recipe's own 1 meV tolerance) -- smaller than the 460 meV seen at the original 50 Å box, but still a real, physically-expected periodic-vs-non-periodic difference on a charged long-range model, not an engine bug.

3. LAMMPS joins the comparison

metawork/lammps (branch metatomic) already ships the ML-METATOMIC package (pair_style / fix / compute metatomic) -- it just wasn't built. Configuring and compiling it (cmake -D PKG_ML-METATOMIC=on , pointing CMAKE_PREFIX_PATH at this venv's own libtorch so LAMMPS links the same torch build metatomic.torch already uses) took a few minutes, most of it spent auto-fetching and compiling matching metatensor-core / metatensor-torch / metatomic-torch C++ releases. hourglass_engines.run_lammps wraps it the same way run_ase / run_ipi wrap their engines: write a .data file and a LAMMPS input script, run lmp , read back energies via a fix print .

LAMMPS's own practical box-size ceiling for LOREM turned out to be tighter than i-PI's: a single 100-step run at DEFAULT_CELL (150 Å, section 1's own pick) didn't finish in 120 s in an isolated process, against ~23-26 s at 50 Å for both i-PI and LAMMPS. So this section's three-engine comparison uses **50 Å**, not DEFAULT_CELL -- the common ground all three engines can actually run in practical time; the 150 Å comparison in section 2 stays ASE/i-PI-only for that reason.

```
In [ ]: from hourglass_engines import run_lammps

THREE_ENGINE_BOX = 50.0
engines3 = {"ASE": run_ase, "i-PI": run_ipi, "LAMMPS": run_lammps}

trajectories3 = {}
for model_name, model_path in model_paths.items():
    trajectories3[model_name] = {}
    for engine_name, run_engine in engines3.items():
        kwargs = {} if engine_name == "ASE" else {"cell": THREE_ENGINE_BOX}
        trajectories3[model_name][engine_name] = run_engine(model_path, atom
```

model	engine	max $ \Delta U $ vs ASE (meV)
sn2-matched-lorem	ASE	0.0000
sn2-matched-lorem	i-PI	459.8445

sn2-matched-lorem	LAMMPS	459.8455
sn2-matched-pet	ASE	0.0000
sn2-matched-pet	i-PI	0.0076
sn2-matched-pet	LAMMPS	0.0153

LAMMPS and i-PI agree with *each other* almost exactly for LOREM (459.8455 vs. 459.8445 meV vs. ASE -- only 0.0010 meV apart) -- independent confirmation that section 1's periodic- image-of-charge explanation is a property of *evaluating this model in any periodic box*, not an artifact of one engine's particular Ewald implementation. **PET matches to noise level in both** (0.0153 meV LAMMPS, 0.0076 meV i-PI).

Which engine is actually fastest?

Repeated 3x on the same PET NVE trajectory (short-range, so box size doesn't confound the comparison) to separate real steady-state cost from one-time warm-up: ASE and i-PI run inside one persistent Python process (so a JIT/compile cost can show up on the first call and disappear after); LAMMPS spawns a brand new `lmp` subprocess on every single call, so it never gets to amortize a warm-up the way the other two can -- and its timings should be flat across trials as a result.

```
In [ ]: import time

speed_rows = []
for trial in range(3):
    row = {"trial": trial}
    for engine_name, run_engine in engines3.items():
        kwargs = {} if engine_name == "ASE" else {"cell": THREE_ENGINE_BOX}
        t0 = time.perf_counter()
        run_engine(model_paths["sn2-matched-pet"], atoms0, ensemble="nve", *
        row[engine_name] = time.perf_counter() - t0
    speed_rows.append(row)
```

trial	ASE (s)	i-PI (s)	LAMMPS (s)
0	2.13	2.13	1.45
1	2.01	1.71	1.45
2	2.13	1.74	1.48

LAMMPS is the fastest engine for PET, consistently (1.45-1.48 s across all 3 trials, vs. i-PI's 1.71-2.13 s and ASE's 2.01-2.13 s) -- and, as expected, **its timings barely move between trials** (no persistent process to warm up), while ASE/i-PI's own numbers settle after their first call. This ordering does not hold for LOREM: there, i-PI and LAMMPS's shared periodic-box cost (section 1 and above) swamps any base per-engine speed difference, and ASE (the only non-periodic engine here) ends up fastest simply by not paying that cost at all.

4. GROMACS joins the comparison

metawork/gromacs (branch metatomic) ships a GMX_METATOMIC module the same way LAMMPS ships ML-METATOMIC . Configuring it needed two local fixes to this fork's own cmake pins, found by just reading the errors:

cmake/gmxManageMetatomic.cmake pinned a stale SHA256 for metatensor-torch-cxx-0.10.0.tar.gz (the real hash didn't match what LAMMPS's own cmake module downloads for the identical file under the identical tag -- copied the working value across), and pinned metatensor-core at 0.1.17, which metatensor-torch 0.10.0 itself refuses to build against (Incompatible versions: we need 0.2.2, but we got 0.1.17 -- bumped to 0.2.3, again matching LAMMPS's own pin). A separate, unrelated GMX_TORCH-gated module (nnpot) also needed GMX_NNPOT=OFF and a working Python_EXECUTABLE to configure cleanly. Topology is inert (masses only, no charges/LJ -- same idea as run_lammps 's pair_coeff , generalized from the upstream recipe's data/topol.top), all forces come from metatomic-active=yes .

GROMACS matches ASE well for PET (0.0177 meV, at the same 50 Å box as section 3).

GROMACS does not work in practical time for LOREM at all: a single 100-step run didn't finish in over 10 minutes even at the *smallest* box tried (20 Å) -- worse than either i-PI's or LAMMPS's own box-size walls (sections 1 and 3), and unlike those two, this isn't box-size-dependent in any way we could observe -- it simply didn't get through 100 steps at any size tried. That's documented in hourglass_engines.py as a limitation, not chased further here; LOREM+GROMACS is left out of the routine comparison.

```
In [ ]: from hourglass_engines import run_gromacs

gmh_ase_t, gmh_ase_e = run_ase(model_paths["sn2-matched-pet"], atoms0, ensemble="nve",
gmh_t, gmh_e = run_gromacs(model_paths["sn2-matched-pet"], atoms0, ensemble="nve",
```

All four engines, PET speed

Same repeated-3x protocol as section 3, now with GROMACS added.

```
In [ ]: engines4 = {**engines3, "GROMACS": run_gromacs}

speed_rows4 = []
for trial in range(3):
    row = {"trial": trial}
    for engine_name, run_engine in engines4.items():
        kwargs = {} if engine_name == "ASE" else {"cell": THREE_ENGINE_BOX}
        t0 = time.perf_counter()
        run_engine(model_paths["sn2-matched-pet"], atoms0, ensemble="nve", *kwargs)
        row[engine_name] = time.perf_counter() - t0
    speed_rows4.append(row)
```

trial	ASE (s)	i-PI (s)	LAMMPS (s)	GROMACS (s)
0	2.13	2.13	1.45	26.67
1	2.01	1.71	1.45	26.87
2	2.13	1.74	1.48	26.95

GROMACS is the slowest engine here, by a wide margin (26.7-27.0 s, vs. LAMMPS's 1.45-1.48 s) -- not because the model evaluation itself is slow (its PET agreement above is fine), but because each `run_gromacs` call is *three* subprocesses (`grompp` , `mdrun` , `gmx energy`), each with its own startup and file-I/O cost, against LAMMPS's one `lmp` call or ASE/i-PI's in-process calls. Like LAMMPS, its timings barely move between trials (no persistent process to warm up) -- the cost is structural (subprocess-per-call), not a JIT/compile artifact.

Takeaways

- **PET matches across engines regardless of box size** (0.0076 meV spread) -- short interaction range means periodicity is never at stake once the box clears the cutoff. All **four** engines now available here (ASE, i-PI, LAMMPS, GROMACS) agree on PET to well under 1 meV.
- **LOREM's discrepancy is a real, physical effect, confirmed by two independent periodic engines:** i-PI and LAMMPS agree with each other to 0.0010 meV at the same box size, while both differ from the non-periodic ASE reference by the same ~460 meV -- this is a property of evaluating a charged, unbounded-range model in *any* periodic box, not one engine's Ewald quirk. Box size trades off against it, but only slowly (section 1): the largest size still practical for i-PI/ASE (150 Å) roughly halves the gap seen at 50 Å (460 meV vs. 228 meV). **Every periodic engine tried has its own box-size (or, for GROMACS, outright practical) wall for LOREM, and each is different:** i-PI reaches 150 Å, LAMMPS only 50 Å, GROMACS not even 20 Å in over 10 minutes -- the same physics, but very different implementation costs across engines.
- **LAMMPS is the fastest engine once built; GROMACS is the slowest, structurally** -- LAMMPS beats ASE and i-PI for the short-range PET model and is immune to the warm-up effects that inflate ASE/i-PI's first call in a fresh process. GROMACS matches everyone on accuracy but pays for three subprocesses (`grompp` / `mdrun` / `gmx energy`) per call, making it ~15-20x slower than LAMMPS here even though its numbers are just as correct. For LOREM, none of this ordering matters: the periodic-box cost swamps every base per-engine speed difference, and GROMACS can't even complete the comparison.
- **A genuine cross-engine NVE parity check for a long-range *and* charged model needs either a neutral (uncharged) test structure, or an engine/library combination whose Ewald cost doesn't scale so steeply with box size** -- simply

"making the box bigger" runs out of computational room long before it runs out of physical justification, for every periodic engine tried here.

- This is the same class of finding as 13 / 14 (lorem-jax 's own non-PBC vs. PBC Ewald convention) and notebook 16 (an exclusion_radius wiring bug in a periodic Ewald port) -- LOREM's long-range term keeps being the place where periodicity assumptions have to be gotten exactly right, across every codebase and engine this series has touched so far.